Section A - Easy

NumPy

1. Create a NumPy array containing odd integers from 11 to 29. Display its number of dimensions and total number of elements.

2. Use NumPy to generate seven equally spaced values between 5 and 35, including both endpoints.

3. Create a 4 x 3 NumPy array filled with 9. Change the element in the third row and second column to 0.

4. Given the following array, calculate the remainder when each element is divided by 5.

```
a = np.array([6, 12, 18, 24])
```

5. Create a NumPy array containing [14, 28, 42, 56, 70, 84]. Extract elements at indices 1, 3, and 5 using slicing.

6. Given np.array([-16, 25, -36, 49]), calculate the absolute value of every element and then its square root.

7. Create a 3 x 2 matrix containing numbers 7 to 12. Display its transpose and state its shape before and after transposition.

8. Calculate the range (maximum minus minimum) and median temperature.

```
temperatures = np.array([18, 22, 27, 31, 24, 19])
```

9. Display distinct values in ascending order and determine how many distinct values are present.

```
values = np.array([4, 9, 4, 2, 9, 7, 2, 4])
```

10. Use np.where() to create a new array containing Fast for speeds of at least 60 and Normal otherwise.

```
speed = np.array([35, 62, 48, 81, 55])
```

Pandas Series

11. Create a Pandas Series representing books issued on Monday, Tuesday, Wednesday, and Thursday, using [24, 31, 19, 27] and the corresponding day labels.

12. Create a Series with labels Laptop, Tablet, Phone and values [45000, 18000, 22000]. Retrieve the tablet price using its label.

13. Display only rainfall measurements greater than 10.

```
rainfall = pd.Series([12, 0, 18, 5, 23])
```

14. Create a Series from the following dictionary. Display its index and values separately.

```
{'Gold': 7200, 'Silver': 95, 'Copper': 850}
```

15. Given a Series ratings containing [3.5, 4.2, 2.8, 4.7], assign Customer Ratings as its name and Review ID as its index name.

16. Check whether X2 exists as an index label and whether X4 is absent.

```
s = pd.Series([11, 22, 33], index=['X1', 'X2', 'X3'])
```

17. Convert the following Series into a dictionary and display the result.

```
stock = pd.Series([16, 28, 12],
                  index=['Keyboard', 'Mouse', 'Monitor'])
```

18. Create a Series from the dictionary below using index order Computer, Hindi, Art, English. Identify the missing entry.

```
{'Hindi': 78, 'English': 84, 'Computer': 91}
```

19. Display the Boolean results of isnull() and notnull().

```
expenses = pd.Series([1200, None, 850, None, 600])
```

20. Predict the result of adding these Series, which have matching labels in different orders.

```
a = pd.Series([8, 15, 22], index=['P', 'Q', 'R'])
b = pd.Series([4, 6, 10], index=['R', 'P', 'Q'])
print(a + b)
```

Section B - Moderate

NumPy

21. A shop records daily sales for three products over four days. Calculate total sales for each day, total sales of each product, and identify the day with the highest combined sales. Rows represent Days 1-4.

```
sales = np.array([
    [120, 150, 180],
    [140, 160, 175],
    [135, 155, 190],
    [150, 170, 200]
])
```

22. Create a 6 x 6 array containing integers 10 to 45. Extract row indices 2 through 5 and column indices 1 through 4, inclusive, using zero-based indexing. Calculate the extracted submatrix mean.

23. Use Boolean indexing to identify weights above 70. Use np.where() to replace weights below 55 with 55 in a new array without changing the original.

```
weights = np.array([48, 62, 75, 81, 54, 69, 90])
```

24. Calculate each product's total cost, the overall bill, and each product's percentage contribution to the bill.

```
quantity = np.array([3, 5, 2, 7])
price = np.array([120, 80, 250, 60])
```

25. Create a 3 x 4 array of your choice. Calculate variance and standard deviation separately for each row and column. Explain axis=0 and axis=1.

26. Choose a 2 x 3 matrix and a 3 x 2 matrix. Perform matrix multiplication. Explain why element-wise multiplication cannot be performed directly between their shapes.

27. Generate 25 random integers from 10 to 99, inclusive. Sort them, extract the five largest values, and determine the number of unique observations.

28. Predict the final contents of a, b, and c. Explain the difference between a view and an independent copy.

```
a = np.array([5, 10, 15, 20, 25, 30])
b = a[1:5]
c = a[1:5].copy()
b[1] = 99
c[2] = 77
```

Pandas Series

29. Add the monthly-revenue Series, identify labels producing missing values, and explain index alignment.

```
dept_a = pd.Series({'Jan': 250, 'Feb': 310, 'Mar': 290})
dept_b = pd.Series({'Mar': 180, 'Jan': 210, 'Apr': 260})
```

30. Identify students with attendance below 75%, display their labels, and count them.

```
attendance = pd.Series(
    [82, 67, 91, 74, 58],
    index=['S101', 'S102', 'S103', 'S104', 'S105']
)
```

31. Predict the Series output, identify the missing value, and explain why Pencil does not appear.

```
inventory = {'Pen': 50, 'Pencil': 80,
             'Eraser': 40, 'Scale': 25}
labels = ['Scale', 'Pen', 'Marker', 'Eraser']
s = pd.Series(inventory, index=labels)
```

32. Predict the result, identify non-missing entries using notnull(), and explain why subtraction is label-based rather than position-based.

```
s1 = pd.Series([100, 200, 300],
               index=['North', 'South', 'East'])
s2 = pd.Series([10, 20, 30],
               index=['South', 'West', 'North'])
result = s1 - s2
```

33. Create a Series of electricity consumption for five households using meaningful labels. Filter consumption above 250 units, convert the filtered Series to a dictionary, and reconstruct a new Series.

34. Predict s[s >= 50], identify missing entries using isnull(), and state the Series and index names.

```
s = pd.Series({'A': 45, 'B': 70, 'C': 55},
              index=['C', 'D', 'A', 'B'])
s.name = 'Performance'
s.index.name = 'Group'
```

Section C - Difficult

NumPy

35. Examination performance analysis: Generate a reproducible 12 x 4 NumPy array of integer test marks from 20 to 100, inclusive, using a fixed seed. Calculate each student's average and each test's mean and standard deviation. Count students passing all four tests (at least 40 in each). Identify students with averages of at least 75 and the test with greatest variation. State your student numbering convention.

36. Matrix transformation and analysis: Construct a 5 x 5 matrix containing integers 21 to 45. Extract the central 3 x 3 matrix, transpose it, and calculate the matrix product of the extracted matrix with its transpose. Calculate row-wise sums of the product and identify its maximum and all positions tied at that maximum. Explain dimensions at every stage.

37. Sales and revenue analysis: Create a 6 x 4 matrix of quantities sold over six days for four products. Use unit prices [150, 240, 90, 320]. Calculate daily revenue, total revenue per product, the highest-revenue day, and the best-selling product by quantity. Reduce prices by 10% for products whose total quantity exceeds 100, then recalculate total revenue without modifying original prices.

38. Random data cleaning and statistics: Generate 100 normal observations with mean 50 and standard deviation 12 using a fixed seed. Count values outside 30-70 and replace them with the nearest boundary using np.where(). Compare original and cleaned mean, variance, standard deviation, minimum, and maximum. Explain the statistical effects of boundary replacement.

Pandas Series

39. Multi-branch financial reconciliation: Combine the three earnings Series by ordinary addition. Explain missing values, identify quarters common to all branches, and calculate their combined earnings. Construct a separate result treating absent branch records as zero and explain how its interpretation differs.

```
branch_a = pd.Series({'Q1': 120, 'Q2': 145, 'Q4': 180})
branch_b = pd.Series({'Q2': 110, 'Q3': 135, 'Q4': 155})
branch_c = pd.Series({'Q1': 95, 'Q3': 125, 'Q4': 140})
```

40. Series data integrity: Preserve the required index order, identify missing student records, and filter valid scores of at least 70. Convert the filtered Series to a dictionary, reconstruct it as a Series, and verify that labelled values match. Explain why missing records should not automatically be zero marks.

```
scores = {'R101': 78, 'R102': 65, 'R104': 92, 'R105': 48}
required = ['R105', 'R103', 'R101', 'R106', 'R104', 'R102']
s = pd.Series(scores, index=required)
```